PRIVACY-ENHANCED SYNTHETIC DATA GENERATION FOR AUTONOMOUS VEHICLE TRAINING

INDIVIDUAL THESIS

Yubitha S
(IT22541284)
Individual component: Secured Dataset Generation
Group ID: R26-CS-010

Report submitted in partial fulfilment of the requirements for the
Bachelor of Science (Honours) Degree in Information Technology
Specialising in Cyber Security

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy (Appendix B). I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Yubitha S

Student ID: IT22541284

Signature: S. Yubitha Date: _03_ / __10__ / 2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Name	Date	Signature
Prof. Anuradha Jayakody	03/10/2026	

Name	Date	Signature
Mr. Dinith Primal	03/10/2026	

ABSTRACT

Autonomous vehicle perception models are trained on large sets of labeled road images. Driving simulators such as CARLA avoid recording real people, but the generated datasets are usually written to disk without any protection. They can be changed, copied or read by anyone who reaches the storage, and they are shared between teams. Existing simulation pipelines focus on realism and give little attention to the security of the data they produce. This thesis presents the Dataset Generation component of a four-part framework for privacy-enhanced synthetic training data. The aim was to build integrity, confidentiality and access control into the generation step and to test them with repeatable checks. The component captured 10,000 CARLA frames in five towns under 20 weather settings, with RGB and depth images and a driving-behavior label for each set. Every file receives a SHA-256 digest, is checked against a role-based access policy, and is encrypted with AES-256 in Galois/Counter Mode. An audit notebook ran five tests on the security classes and scanned the real capture folder. All five tests passed. Decryption restored the data exactly, 30 encryptions of the same input gave 30 different nonces and ciphertexts, the chi-square statistic of the ciphertext was 272.59 against a 5% critical value of about 293.25, 50.0% of ciphertext bits differed between two encryptions, and a modified file was rejected with a MAC check failure. All 6,093 encrypted files in the scanned folder had no readable header. The cost was 32 bytes per file. Analysis of the dataset showed even weather coverage but unbalanced driving behaviors, with Vehicle_Turning at only 1.3% of the sets. The work shows that a synthetic dataset can be protected at the moment of creation at small cost, and that a secure dataset still needs good scene design to be useful.

Keywords: synthetic data, autonomous vehicles, CARLA, authenticated encryption, role-based access control

ACKNOWLEDGEMENTS

I would like to thank my supervisor, Prof. Anuradha Jayakody, and my co-supervisor, Mr. Dinith Primal, for their guidance and feedback throughout this research. Their advice helped me focus the work on a security layer that I could build and test within the project timeline.

I also thank my team members, Priyadarshi S.A.S.D. (IT22309556), Apeksha M.K.S.R (IT22110220) and Wijesinghe R.D.P.T. (IT22066916), for their support and cooperation. Having my dataset used and checked by the other components helped me improve many design decisions in this thesis. I thank the members of the progress review panels for their questions. Finally, I thank the developers of CARLA and PyCryptodome, whose open-source tools made this work possible.

TABLE OF CONTENTS

1 INTRODUCTION **1**
1.1 Overall project context 1
1.2 Individual component context 2
1.3 Problem statement 2
1.4 Research gap 2
1.5 Aim and objectives 2
1.6 Research questions or hypotheses 3
1.7 Scope limitations and assumptions 3
1.8 Individual contribution 4
1.9 Thesis structure 4
2 LITERATURE REVIEW AND RELATED WORK **5**
2.1 Review method and scope 5
2.2 Domain and theoretical foundations 5
2.3 Related research 6
2.4 Existing systems products and techniques 7
2.5 Gap synthesis and conceptual framework 7
3 RESEARCH METHODOLOGY **9**
3.1 Research design 9
3.2 Data participants or study materials 9
3.3 Experimental or study procedure 10
3.4 Measures and evaluation metrics 11
3.5 Data analysis 12
3.6 Ethics privacy security and responsible AI 13
3.7 Reproducibility and research artefacts 13
4 PRODUCT AND SOLUTION DESIGN **14**
4.1 Requirements 14
4.2 Component architecture within the complete solution 14
4.3 Design alternatives and decisions 15
4.4 Detailed design 16
4.5 Standards and professional practice 18
5 IMPLEMENTATION INTEGRATION AND TESTING **20**
5.1 Development environment and technology stack 20
5.2 Component implementation 20
5.3 Integration with the other components 22
5.4 Verification and software testing 22
5.5 Deployment and operation 23
6 RESULTS AND VALIDATION **25**
6.1 Research results 25
6.2 Product and technical results 28
6.3 User or stakeholder validation 31
6.4 Integration results 32
6.5 Summary of findings 32
7 DISCUSSION **34**
7.1 Interpretation of findings 34
7.2 Comparison with literature baselines and existing solutions 34

7.3 Research contribution 35
7.4 Product contribution 35
7.5 Limitations and threats to validity 35
7.6 Implications 36
8 COMMERCIALISATION AND SUSTAINABILITY 38
8.1 Value proposition and target users 38
8.2 Market and competitive position 38
8.3 Business or sustainability model 38
8.4 Scalability intellectual property and compliance 39
8.5 Contribution to the Sustainable Development Goals 39
9 CONCLUSION AND FUTURE DIRECTIONS 41
9.1 Conclusion 41
9.2 Achievement of objectives 41
9.3 Individual research and product contributions 41
9.4 Future directions 42
REFERENCES 43
GLOSSARY 45
APPENDICES 46
Appendix A Extended results 46
Appendix B Academic integrity and AI use disclosure 47

LIST OF FIGURES

Figure 1.1: Overall system overview of the R26-CS-010 framework 1
Figure 2.1: Conceptual framework of the Dataset Generation component 8
Figure 3.1: Research design and experimental procedure of the component 9
Figure 4.1: Group pipeline architecture and the position of Dataset Generation 15
Figure 4.2: Internal data flow of the Dataset Generation component 16
Figure 4.3: Layout of an encrypted dataset file 16
Figure 6.1: Results of audit Tests 2, 3 and 4 and the real folder scan 27
Figure 6.2: Security property dashboard (5 of 5 tests passed) 28
Figure 6.3: Weather distribution per town (number of captured sets) 29
Figure 6.4: Global dataset behavior distribution 29
Figure 6.5: Behavioral density heatmap (town versus event) 30

LIST OF TABLES

Table 2.1: Existing systems and techniques compared with the needs of the component 7
Table 2.2: Research gaps addressed by the Dataset Generation component 7
Table 2.3: STRIDE threat classes and their coverage in the component 8
Table 3.1: Town and weather settings of the captured dataset 10
Table 3.2: Security audit test plan 11
Table 3.3: Measures and evaluation metrics 12
Table 3.4: Reference values for the statistical audit tests 12
Table 4.1: Functional requirements of the Dataset Generation component 14
Table 4.2: Design alternatives and decisions 15
Table 4.3: RBAC role and permission matrix 17
Table 4.4: Threat model and corresponding controls 17
Table 4.5: Security parameters of the implementation 17
Table 4.6: Fields of the town metadata files 18
Table 5.1: Development environment and technology stack 20
Table 5.2: Files exchanged by the Dataset Generation component 22
Table 5.3: Verification of the functional requirements 23
Table 5.4: Test cases of the security audit 23
Table 6.1: Round-trip correctness results 25
Table 6.2: Nonce uniqueness results 25
Table 6.3: Ciphertext byte distribution results 26
Table 6.4: Bit difference results 26
Table 6.5: Tamper detection results 27
Table 6.6: Real capture folder validation results 27
Table 6.7: Summary of audit results 28
Table 6.8: Distribution of behavior labels over the dataset 29
Table 6.9: Behavior labels by town (sets and share of the 2,000 sets of each town) 30
Table 6.10: Diversity of behavior labels by town 31
Table 6.11: Cost of the security layer 31
Table 6.12: Integration results reported by the other components 32
Table 7.1: Limitations, their effect and suggested remedies 36
Table 8.1: Project budget estimate from the proposal (whole group) 39
Table 9.1: Achievement of objectives 41
Table A.1: Raw values of the security audit 46
Table A.2: First five records of the Town01 metadata file 46
Table A.3: Number of sets per behavior label and town 47
Table A.4: Expected outcome of each role and permission pair 47
Table A.5: Steps of the security audit notebook 47

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
AI	Artificial Intelligence
AV	Autonomous Vehicle
CARLA	Car Learning to Act (open-source driving simulator)
CBC	Cipher Block Chaining
CSV	Comma-Separated Values
DP	Differential Privacy
FR	Functional Requirement
GCM	Galois/Counter Mode
GDPR	General Data Protection Regulation
JSON	JavaScript Object Notation
KITTI	Karlsruhe Institute of Technology and Toyota Technological Institute (dataset)
MAC	Message Authentication Code
mAP	Mean Average Precision
NIST	National Institute of Standards and Technology
PNG	Portable Network Graphics
RBAC	Role-Based Access Control
RGB	Red, Green, Blue
RQ	Research Question
SDG	Sustainable Development Goal
SHA	Secure Hash Algorithm

1 INTRODUCTION

1.1 Overall project context

Autonomous vehicles use machine learning to detect objects, follow lanes and decide how to drive. These models need very large amounts of labeled sensor data. Public datasets such as KITTI [1] and the Waymo Open Dataset [2] were built by driving sensor-equipped cars on public roads. The recordings contain personal information. Cameras capture faces and number plates, and repeated routes can show where people live and work. Under the General Data Protection Regulation (GDPR), recordings of identifiable people are personal data, and Article 32 requires appropriate technical measures to keep such data secure [3].

A common answer is to generate the training data in a driving simulator. CARLA is a widely used open-source simulator that provides towns, traffic, weather and virtual sensors [4], and it was used to build large synthetic datasets such as SHIFT [5]. Synthetic data avoids recording real people. It does not, however, remove the need for security. A generated dataset is still a valuable file collection. It can be copied, read, altered or deleted by anyone who can reach the storage, and datasets are increasingly shared between universities, suppliers and manufacturers. Reviews of autonomous driving security list data tampering and unauthorized access among the main threats [6], [7].

Project R26-CS-010 addresses these risks with a privacy-enhanced synthetic data generation framework made of four components. Privacy Integration (IT22309556) applies and tests differential privacy where real information enters the pipeline. Dataset Generation (IT22541284) captures CARLA frames and protects every file with SHA-256 hashing, AES-256 encryption and role-based access control (RBAC). Utility Evaluation (IT22110220) checks the data for tampering and measures how useful it is for training a detector. Risk and Compliance (IT22066916) verifies the privacy controls with independent attacks and maps the evidence to GDPR and NIST requirements. Figure 1.1 shows how the four components connect.

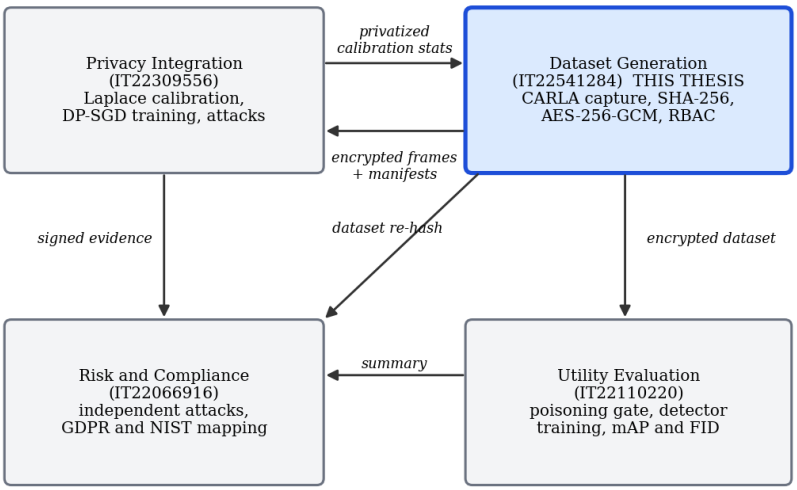


Figure 1.1: Overall system overview of the R26-CS-010 framework

1.2 Individual component context

This thesis covers the Dataset Generation component. It is the part of the framework that produces the data. It runs CARLA in five town maps under several weather conditions, captures RGB and depth images, labels each captured set with a driving behavior, and protects every file before it leaves the component. Its output is an encrypted dataset of 10,000 frames, together with a metadata file for each town and a manifest of SHA-256 digests.

The component sits at the start of the data flow. Before capture, it receives privatized calibration statistics from Privacy Integration, which configure the simulator. After capture, it hands the encrypted frames and manifests to Privacy Integration for differential-privacy training, to Utility Evaluation for the tampering check and detector training, and to Risk and Compliance for independent re-hashing [8]. Its output is therefore the input of the other three components, and a weakness in it would weaken all of them.

1.3 Problem statement

Simulation-based data generation frameworks concentrate on realism, sensor fidelity and variety. Protection of the generated files is usually treated as a later step, and where it exists it is applied after generation [6], [7], [9]. Between capture and protection, raw images sit on disk with no integrity check, no encryption and no restriction on who may open them. Techniques such as blurring or anonymizing images protect what is shown in a picture, but they do not stop a file being modified, do not stop it being read, and do not control who handles the raw output.

As a result, a team that shares a synthetic dataset cannot show that the files are unchanged since capture, that they are unreadable without the key, or that only authorized roles handled them. Poisoned or corrupted training data can also degrade a perception model without being noticed [10], [11]. A security layer built into the generation step, and tested in a way others can repeat, is therefore needed.

1.4 Research gap

Earlier work leaves four gaps that matter for this component. First, studies on synthetic driving data focus on realism, diversity and domain adaptation, and rarely treat the security of the generation pipeline [5], [12]. Second, security reviews of autonomous driving describe threats and defenses in general, but few evaluate a working control inside a data generation pipeline [6], [9]. Third, privacy methods such as differential privacy reduce what data reveals, but they do not secure the stored files against tampering or unauthorized reading [13]. Fourth, encryption is often described but rarely checked with measurable tests. A design may use a strong cipher and still fail through nonce reuse or an unauthenticated mode [14], [15]. Section 2.5 explains these gaps in more detail.

1.5 Aim and objectives

The aim of this research is to build a security layer into the synthetic data generation pipeline and to show, with repeatable tests, that it protects the dataset. The project proposal set three measurable targets for this component. Every generated file must be hashed, encrypted and covered by an access policy. All security audit tests must pass. No encrypted file may be readable without the key. The specific objectives are:

1. To configure CARLA to capture RGB and depth frames across several town maps and weather conditions.
2. To generate a diverse, labeled dataset and to describe its coverage of towns, weather and driving behaviors.
3. To compute a SHA-256 digest for every captured file and to keep a manifest for later verification.
4. To encrypt every file with AES-256 authenticated encryption before storage or transfer.
5. To enforce least-privilege access with an RBAC gateway for three pipeline roles.
6. To validate the security layer with an independent audit covering correctness, nonce uniqueness, ciphertext randomness, ciphertext difference and tamper detection.
7. To integrate the component with the other three components and to consume the privatized calibration statistics.

1.6 Research questions or hypotheses

The main research question of this component is:

RQ: Can integrity hashing, authenticated encryption and role-based access control be built into the CARLA data generation pipeline so that the generated dataset is protected at the moment of creation, and can that protection be verified by tests?

The question was split into three sub-questions:

- **RQ1:** Can every captured frame be hashed, encrypted and covered by an access policy at capture time, without changing how CARLA is used?
- **RQ2:** Does the encryption layer pass tests for correctness, semantic security, ciphertext randomness and tamper detection?
- **RQ3:** Is the generated dataset diverse enough across towns, weather and behaviors to be useful, and what does the security layer cost in storage and time?

The working hypothesis was that the encrypted output would decrypt exactly, that identical inputs would never give identical ciphertexts, that the ciphertext would look statistically random, and that any change to an encrypted file would be rejected.

1.7 Scope limitations and assumptions

The component covers the protection of dataset files at generation and at rest. It does not cover differential privacy, model training or attack testing, which belong to the other components. It does not protect the simulation computer, data held in memory, or the encryption key itself beyond the limits discussed in Section 7.5. Network security and user authentication are also outside the scope.

The work makes four assumptions. The simulation computer and the pipeline code are trusted. The storage that holds the dataset, and the people who can reach it, are not fully trusted. All frames come from CARLA, so no real person, vehicle or place is recorded. The calibration source values used by Privacy Integration were reference statistics measured from CARLA runs, not statistics from a real dataset such as Waymo [8].

1.8 Individual contribution

The author designed and built the whole Dataset Generation component. The main contributions are:

- A multi-town CARLA capture pipeline that produced 10,000 frames across five towns and 20 town and weather settings, with a driving-behavior label for every frame.
- A `SecurityEngine` class that computes SHA-256 digests in blocks and encrypts every file with AES-256 in Galois/Counter Mode (GCM).
- An `RBACGateway` class with deny-by-default permissions for the Simulation Engineer, Privacy Engineer and ML Researcher roles.
- A design change from the proposed CBC mode to authenticated encryption, supported by testing.
- A self-contained audit notebook with five tests and a scan of the real capture folder, which can be repeated without CARLA.
- A dataset analysis of weather, behavior and town coverage, and the metadata and interfaces that connect the component to the other three components.

1.9 Thesis structure

Chapter 2 reviews related work and builds the conceptual framework. Chapter 3 explains the research design, data, procedure and metrics. Chapter 4 presents the requirements and the design of the component. Chapter 5 covers its implementation, integration and testing. Chapter 6 reports the results, and Chapter 7 discusses them. Chapter 8 considers commercialization and sustainability, and Chapter 9 concludes the thesis.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review covered four topics that the component depends on. These were simulation-based synthetic data for autonomous driving, the security threats to autonomous driving data, the cryptographic and access-control methods used to protect stored data, and the link between security and privacy for synthetic data. Sources are mainly peer-reviewed papers and standards, including CVPR, USENIX, EUROCRYPT and IEEE journals, together with NIST publications. Security surveys were checked against their published records. Work was included if it described a method the component uses, tested synthetic driving data, or analyzed threats to autonomous driving data.

2.2 Domain and theoretical foundations

Synthetic data for autonomous driving

Real benchmark datasets such as KITTI [1] and Cityscapes [16] gave the field labeled urban scenes, but real collection is costly and raises privacy concerns. Virtual worlds were therefore used to produce labeled data cheaply. Examples include SYNTHIA [17] and the CARLA simulator [4], with AirSim [18] offering a similar platform. CARLA provides several town maps, traffic, pedestrians, weather presets and sensors such as RGB cameras and depth cameras, with automatic labels. Synthetic data also lets researchers create rare situations that are hard or unsafe to record on real roads. Domain randomization is one way to reduce the gap between simulated and real images [19], and a survey of synthetic data for deep learning describes both the benefits and the open problems [12]. The SHIFT dataset, built entirely in CARLA, shows that simulation-based generation can reach large scale [5].

Threats to autonomous driving data

Gao et al. surveyed the security of autonomous driving and described attack surfaces for sensors, operating systems, control systems and vehicle communication, with data integrity and confidentiality among the security properties examined [6]. Liao et al. discussed data security from technical, legal and ethical sides and pointed out that the pace of law lags behind technology and that breaches by internal personnel are a real risk [7]. Lippi et al. reviewed attack vectors and defenses, including privacy-preserving architectures [9]. Xie et al. surveyed the privacy of autonomous vehicles and the protection methods available [13]. Together, these works agree that stored AV data needs integrity, confidentiality and access control. A synthetic data pipeline is no exception.

Training data can also be attacked directly. Poisoning attacks change training examples so that a model learns the wrong behavior [10], and backdoor attacks hide a trigger in the data supply chain [11]. These attacks do not need access to the model. They only need the ability to change files, which is why detecting file changes matters for a dataset that is stored and shared.

Hashing, encryption and authenticated encryption

SHA-256 is a hash function from the SHA-2 family that maps any input to a 256-bit digest [20]. A change of one bit in the input gives a completely different digest, and finding two inputs with the same digest is not feasible. Storing the digest of each file at capture time creates a fingerprint that can later be recomputed and compared. A hash shows whether a file changed. It does not hide the content and it does not prove who made it.

The Advanced Encryption Standard (AES) is the standard block cipher. It works on 128-bit blocks with keys of 128, 192 or 256 bits [21]. A mode of operation is needed to encrypt files longer than one block. The choice of mode matters. Cipher Block Chaining (CBC) gives confidentiality only. A modified ciphertext can decrypt to damaged data without an error, and CBC implementations that reveal padding errors can be attacked [22]. Detecting changes therefore needs a separate mechanism, and combining encryption and authentication correctly is a known source of mistakes [23].

Galois/Counter Mode (GCM) is an authenticated-encryption mode. It encrypts with counter mode and computes an authentication tag over the ciphertext with a Galois-field function [24], [25]. During decryption the tag is checked first, and any change to the ciphertext, nonce or tag makes decryption fail. GCM has one strict rule: a nonce must never be reused under the same key. Reuse breaks both confidentiality and authenticity, and practical forgery attacks on GCM through nonce reuse have been shown in real systems [14], [15]. For this reason, nonce uniqueness is a property that must be tested.

Access control

Role-based access control (RBAC) assigns permissions to roles instead of to individual users, and users receive permissions through the roles they hold [26]. It supports two classic design principles. Least privilege gives each role only the permissions it needs for its duties, and deny by default refuses anything that is not explicitly allowed [27]. In a data pipeline, RBAC separates the people and programs that create data, those that process it and those that only use the result.

2.3 Related research

Synthetic data generation. Most work on synthetic driving data aims to make the data realistic or to close the gap to real images. Studies such as SHIFT [5] and SYNTHIA [17] describe the simulator configuration and the labels, but they do not describe how the generated files are protected. The CARLA paper [4] does not discuss integrity checks, encryption or access control for the data the simulator produces.

Privacy of synthetic data. Stadler et al. showed that synthetic data does not protect the records it came from unless a formal privacy mechanism is used [28]. Differential privacy gives such a mechanism [29], [30]. This protects the information in the data. It is a different control from

the one studied here. A privacy-protected dataset can still be modified or copied if its files are not secured, so the two controls are complementary.

Security of autonomous driving data. The security surveys [6], [7], [9] describe the threat landscape in broad terms. They recommend encryption, integrity protection and access control, but they do not test these controls in a synthetic data pipeline. Liao et al. also stress that legal and ethical duties around data security are growing [7], which supports building protection into the pipeline from the start.

Evaluating encryption in practice. Cryptographic standards define what a correct implementation must do [21], [25], and statistical test suites exist to examine randomness [31]. Studies of real failures show that the mode and the nonce handling cause most problems, not the cipher itself [15], [22]. A practical evaluation of a data pipeline should therefore check correctness, nonce uniqueness, randomness and tamper detection, which is what this thesis does.

2.4 Existing systems products and techniques

Several tools and practices cover parts of the problem. Table 2.1 compares them with the needs of this component.

Table 2.1: Existing systems and techniques compared with the needs of the component

System or technique	What it provides	Limitation for this work
CARLA, AirSim and similar simulators [4], [18]	Labeled synthetic sensor data	No protection of generated files; no integrity record or access policy
Public datasets such as KITTI, Waymo and SHIFT [1], [2], [5]	Large benchmark data and labels	Distributed as plain files; protection is left to the user
Image blurring and anonymization	Removes visible identifiers from images	Does not stop tampering or unauthorized reading of files
Differential privacy methods [29]	Formal limit on what a model reveals	Protects information content, not file integrity or access
Disk or cloud storage encryption	Encrypts data at rest	Applied after generation; no per-file integrity manifest or role policy for the pipeline

Disk and cloud encryption are the closest practical alternative. They protect stored data, but they work on the storage, not on the generation step. They do not record a digest for each captured frame, they do not apply a role policy to pipeline operations, and they give no per-file evidence that can be shared with another team. The component described here applies the controls per file at the moment of capture.

2.5 Gap synthesis and conceptual framework

Table 2.2 summarizes the gaps that this component addresses and the approach taken for each.

Table 2.2: Research gaps addressed by the Dataset Generation component

Gap	Limitation of existing work	Approach in this component
Pipeline security	Synthetic data work focuses on realism; security is left to later steps [5], [6]	Controls built into capture: hash, encrypt, check role
Tamper detection	Unauthenticated modes such as CBC cannot detect changes alone [22]	AES-256-GCM with a per-file authentication tag, plus a SHA-256 manifest
Measured evidence	Encryption is described but rarely tested [15]	Five-test audit and a scan of the real capture folder
Least privilege	Access control is recommended but not applied to pipeline roles [7]	RBAC gateway with deny-by-default permissions for three roles

These gaps lead to the conceptual framework in Figure 2.1. Each threat to the dataset has a matching control. Each control produces evidence, and a separate check tests that evidence. Tampering is met with a SHA-256 manifest and the GCM tag. Disclosure is met with AES-256-GCM encryption. Privilege misuse is met with the RBAC gateway. Protection is claimed only when the evidence from the audit agrees.

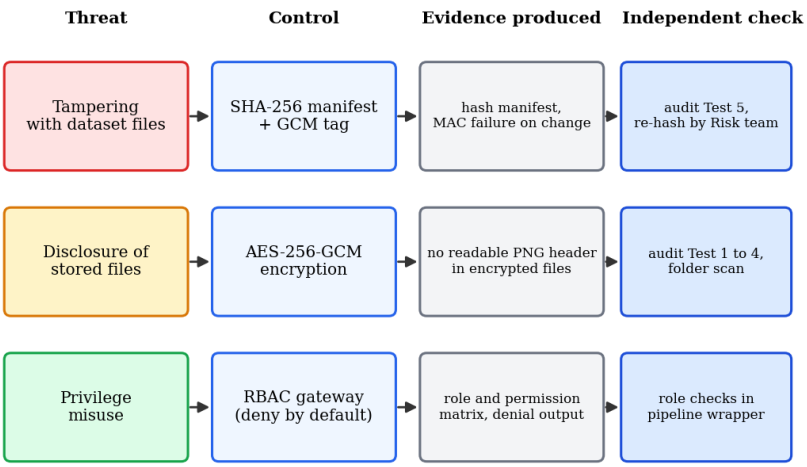


Figure 2.1: Conceptual framework of the Dataset Generation component

The conceptual framework can be checked against a standard threat classification. STRIDE groups threats into spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privilege [32]. It helps a designer see which classes of threat a design covers and which it leaves open. Table 2.3 applies it to the dataset pipeline. It shows that the component covers tampering, information disclosure and elevation of privilege well, covers spoofing only in part, and leaves repudiation and denial of service open. These open items are carried into the limitations in Section 7.5 and the future work in Section 9.4.

Table 2.3: STRIDE threat classes and their coverage in the component

STRIDE class	Example in a dataset pipeline	Coverage in this component
Spoofing	A user claims a role they do not hold	Partly: roles are checked, but users are not authenticated
Tampering	Images or labels are changed in storage	Covered: SHA-256 manifest and GCM authentication tag
Repudiation	Nobody can show who read or changed a file	Not covered: no persistent access log
Information disclosure	Dataset files are copied from storage	Covered: AES-256-GCM encryption, plaintext deleted
Denial of service	Files are deleted or the storage is filled	Not covered: the manifest shows missing files but does not prevent loss
Elevation of privilege	An ML user reads raw simulation output	Covered: deny-by-default RBAC policy

3 RESEARCH METHODOLOGY

3.1 Research design

The research followed a design science approach with quantitative evaluation. A security artefact was built first and then tested in controlled experiments. The design had four parts:

- **Build and generate.** The capture pipeline and the security layer were built, and the dataset was generated in five CARLA towns.
- **Isolated testing.** The security classes were copied into an audit notebook that runs without CARLA. Each test checks one property and changes only one factor.
- **Real-data check.** After the controlled tests, the notebook scanned the real encrypted capture folder to confirm that real output is protected.
- **Integration check.** The other three components used the dataset and checked it independently, so problems in the interface would show up.

Figure 3.1 shows the research process from the problem to the conclusions. The audit results fed back into the design. In particular, the proposal specified CBC, and the encryption mode was changed to GCM because CBC cannot detect modified files on its own.

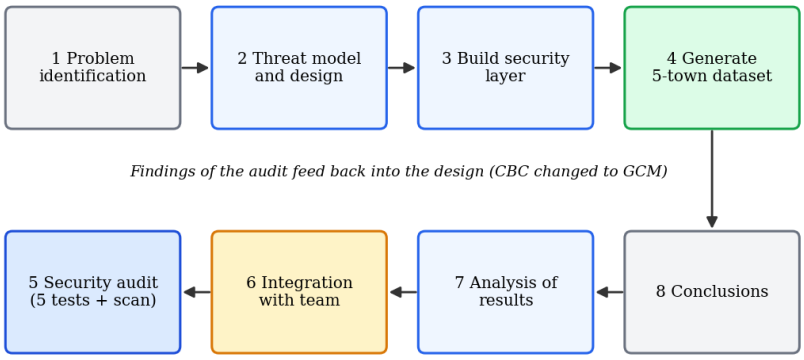


Figure 3.1: Research design and experimental procedure of the component

3.2 Data participants or study materials

No human participants took part in this study. The study material was synthetic data generated in CARLA.

Towns and weather. The dataset was captured in five CARLA town maps, Town01 to Town05. Each town was captured under four weather settings, and each setting produced 500 captured sets. Each town therefore has 2,000 sets and the full dataset has 10,000 frames. There are 20 town and weather settings and 18 different weather labels, because Heavy_Rain is used in Town01 and Town02 and Clear_Noon is used in Town03 and Town04. Table 3.1 lists the settings. Frames were captured at 800 x 600 pixels [8].

Table 3.1: Town and weather settings of the captured dataset

Town	Weather settings (500 sets each)
Town01	Clear_Day, Overcast, Heavy_Rain, Stormy_Night
Town02	Clear_Morning, Heavy_Rain, Dense_Fog, Sunset_Glare
Town03	Clear_Noon, Urban_Storm, Night_Rain, Urban_Dawn
Town04	Clear_Noon, Highway_Sunset, Mountain_Mist, High_Wind_Rain
Town05	High_Contrast_Noon, Heavy_Urban_Storm, Clear_Night_City, Morning_Mist_Urban

Town maps. The five towns differ in layout. According to the CARLA documentation [33], Town01 is a small town with a river and several bridges, Town02 is a small residential town, Town03 is a larger urban map with a roundabout and roads at different heights, Town04 is a small town in the mountains with a long highway, and Town05 is an urban map with a square grid of cross junctions and multilane roads. These differences give the dataset variety in road structure as well as in weather.

Captured sets and metadata. Each captured set consists of an RGB image and a depth image of the same frame. Every town has a metadata file that lists, for each set, the encrypted RGB file name, the encrypted depth file name, the weather label, a driving-behavior label, the town and a security field. In the first records of Town01, the frame numbers in the file names increase by 16, so frames were taken at a regular interval. The security field reads `SHA256+AES256` for every record.

Behavior labels. Each set carries one of five driving-behavior labels: Standard_Flow, Heavy_Braking, Intersection_Nav, High_Speed_Cruise and Vehicle_Turning. They describe what the ego vehicle was doing when the frame was captured. The labels allow the dataset to be analyzed for coverage and balance.

Calibration statistics. The simulator is configured from seven aggregate statistics: vehicle and pedestrian density per kilometer, average vehicle and pedestrian speed, intersections per kilometer, and the weather and time-of-day distributions. The source values were reference statistics that this component measured from CARLA runs. They passed through the Laplace mechanism in Privacy Integration, and only the privatized values were used to configure the simulator [8].

Security test material. The audit used files of pseudo-random bytes between 4 KB and 200 KB, created with fixed seeds so that every test can be repeated. The final scan used the real encrypted capture folder.

3.3 Experimental or study procedure

The procedure had six steps.

1. **Configuration.** CARLA was started in the chosen town. The weather preset and, for Town01, the privatized calibration values were applied.

2. **Capture.** The ego vehicle drove through the town and RGB and depth images were recorded at a fixed frame interval.
3. **Protection.** For every captured file, the pipeline computed a SHA-256 digest, checked the role against the RBAC policy, encrypted the file with AES-256-GCM, wrote the `.enc` file and removed the plaintext.
4. **Metadata.** One row was written to the town metadata file for each captured set, with the weather, behavior label, town and security fields.
5. **Audit.** The audit notebook ran five tests on the security classes and then scanned the real encrypted folder.
6. **Analysis and hand-over.** The metadata of all five towns was analyzed for coverage, and the encrypted dataset and manifests were passed to the other components.

The five audit tests are listed in Table 3.2. Each one was designed to fail if a specific weakness exists.

Table 3.2: Security audit test plan

No.	Test	What it checks	Pass criterion
1	Round-trip correctness	Decryption restores the original data	Recovered bytes identical to the original
2	Nonce uniqueness	Same input never gives the same output	30 encryptions give 30 distinct nonces and ciphertexts
3	Byte distribution	Ciphertext looks random	Chi-square statistic below the 5% critical value (df = 255)
4	Bit difference	Near-identical inputs give unrelated ciphertexts	About 50% of ciphertext bits differ
5	Tamper detection	A changed file is rejected	MAC check fails and no output file is written

For Test 1, a 4 KB file was encrypted and decrypted. For Test 2, one plaintext was encrypted 30 times with the same key. For Test 3, a 200,000-byte file was encrypted and the frequency of each of the 256 byte values in the ciphertext was counted. For Test 4, two 4 KB inputs that differ in exactly one bit were encrypted and their ciphertexts compared bit by bit. For Test 5, one byte of an encrypted file was changed with an XOR of 0xFF and decryption was attempted.

3.4 Measures and evaluation metrics

Table 3.3 lists the measures used in the evaluation.

Table 3.3: Measures and evaluation metrics

Measure	Definition	Purpose
Round-trip match	Recovered bytes equal original bytes	Correctness of encryption
Unique nonces and ciphertexts	Count of distinct values in 30 runs	Semantic security
Chi-square statistic	Sum over 256 byte values of (observed - expected)^2 / expected	Randomness of ciphertext
Bit difference	Share of ciphertext bits that differ between two encryptions	Unrelatedness of ciphertexts
Tamper result	Whether a modified file is accepted or rejected	Authenticity (GCM tag)
Readable-header count	Encrypted files starting with the PNG signature	Protection of real output
Encryption and decryption time	Milliseconds for a 4 KB file, including file input and output	Cost of the security layer
Storage overhead	Extra bytes per encrypted file	Cost of the security layer
Coverage counts	Sets per town, weather and behavior label	Usefulness and diversity of the dataset

3.5 Data analysis

The security results are reported as direct measurements. The chi-square statistic is compared with the 5% critical value for 255 degrees of freedom, which is about 293.25, and a p-value is computed from it. Tests 1, 2 and 5 are pass or fail tests. Test 4 is judged against the 50% value that unrelated random data would give.

The chi-square statistic for Test 3 is calculated as follows. Let N be the number of ciphertext bytes, and let O(i) be the number of times the byte value i appears, for i from 0 to 255. If the bytes are uniform, each value is expected E = N / 256 times. The statistic is the sum over all 256 values of (O(i) - E)^2 / E. For N = 200,000 the expected count is E = 781.25. When the bytes are random, the statistic follows a chi-square distribution with 255 degrees of freedom. Its mean is 255 and its standard deviation is about 22.6. A single byte count then has a standard deviation of about 27.9. Table 3.4 lists the reference values against which the results are compared.

Table 3.4: Reference values for the statistical audit tests

Quantity	Reference value for random data
Expected count per byte value (N = 200,000)	781.25
Standard deviation of one count	about 27.9
Mean of the chi-square statistic (df = 255)	255
Standard deviation of the statistic	about 22.6
5% critical value (df = 255)	about 293.25
Expected share of differing bits between two unrelated ciphertexts	50% (16,384 of 32,768 bits)
Standard deviation of the differing-bit count	about 90.5 bits

The dataset was analyzed with pandas, matplotlib and seaborn. The metadata files of the five towns were loaded into one table and checked for the total number of records. Counts were then drawn as a weather chart per town, a bar chart of the behavior labels with percentages, and a heatmap of behavior against town. These analyses show what the dataset covers and where it is unbalanced.

3.6 Ethics privacy security and responsible AI

All images in this work were generated inside the CARLA simulator, so no real person, vehicle or place was recorded and no personal data was collected. The component still treats the dataset as sensitive, because the aim of the project is to build a pipeline that would be safe to use with real data.

The component does not claim legal compliance. It produces technical evidence, which is a hash manifest, encrypted files and test results. A data controller can use this evidence when deciding whether a dataset meets its duties under GDPR Article 32 [3]. Known weaknesses, such as the limits of key handling, are stated in this thesis and not hidden. During the project, a code review of the group repository found encryption and signing keys written in source files before the repository was published. The keys were replaced, and the group added a check that fails if an old key appears in the source again [8].

Generative AI was used to help prepare this thesis. The use is described in Appendix B. The author remains responsible for the design, the experiments, the numbers and the conclusions.

3.7 Reproducibility and research artefacts

The audit notebook contains the same `SecurityEngine` and `RBACGateway` source code as the pipeline. It creates its own test files from fixed random seeds and writes the numeric results to a JSON file, so another person can repeat all five tests without CARLA or the dataset. The dataset analysis notebook reads the town metadata files and regenerates the three coverage figures. The main research artefacts are the capture and security code, the encrypted dataset with its manifests and metadata files, the audit notebook and its results file, and the analysis notebook.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements came from the project proposal, the needs of the other three components and the stakeholders named in the proposal. These were autonomous vehicle researchers and AI developers, who need training data they can trust, simulation engineers who run the generator, cybersecurity engineers who audit the controls, and companies that want to share synthetic data safely. Table 4.1 lists the functional requirements of this component.

Table 4.1: Functional requirements of the Dataset Generation component

ID	Requirement	Source
FR-01	Generate driving scenes with CARLA in several towns and weather settings	Proposal FR1, FR2
FR-02	Capture RGB and depth images and label each set	Proposal FR3
FR-03	Compute a SHA-256 digest of every generated file and keep a manifest	Proposal FR4
FR-04	Encrypt every file with AES-256 before storage or transfer	Proposal FR5
FR-05	Enforce role-based access on pipeline operations	Proposal FR6
FR-06	Store the dataset in a structured format with metadata	Proposal FR8
FR-07	Accept privatized calibration statistics to configure the simulator	Needs of Privacy Integration
FR-08	Reject modified encrypted files without writing output	Threat model (tampering)

The non-functional requirements were as follows:

- **Security.** Encrypted files must be unreadable without the key, and changes must be detected.
- **Performance.** Hashing and encryption must not noticeably delay capture.
- **Testability.** The security code must be testable without the simulator.
- **Interoperability.** Other components must be able to decrypt the files and read the metadata and manifests.
- **Scalability.** The design must support more towns, weather settings and sensors without changing the security layer.

4.2 Component architecture within the complete solution

Figure 4.1 shows where the component sits in the group pipeline. It works at the start of the data flow. It receives calibration_stats.json from Privacy Integration. It runs the capture and protection pipeline in the five towns. It then provides the encrypted frames, the manifests and the metadata files to the other three components. Because every file is encrypted and listed in a manifest, each receiving component can check the data before using it.

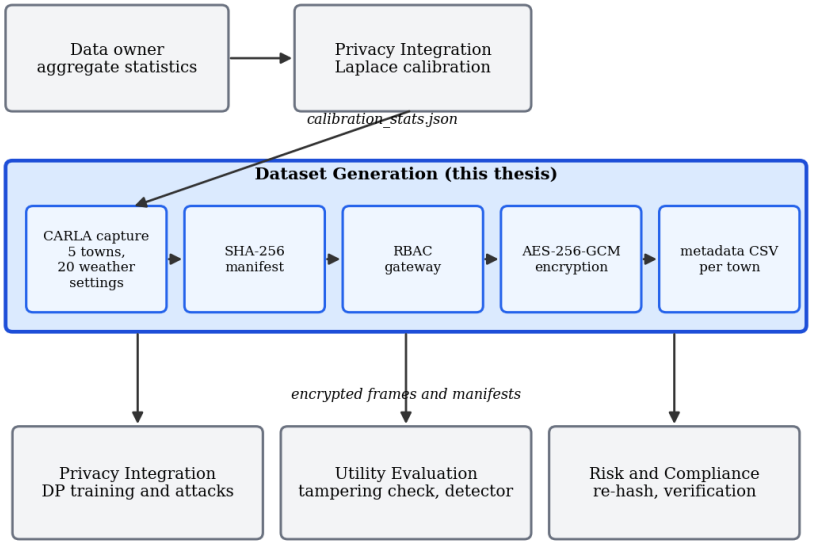


Figure 4.1: Group pipeline architecture and the position of Dataset Generation

This position gives the component a clear boundary. The encrypted dataset and its manifests are the only path from the simulator to the rest of the project. All access to raw output passes through the RBAC check. The boundary can therefore be audited in one place.

4.3 Design alternatives and decisions

Table 4.2 lists the main design choices, the alternatives that were considered and the reason for each choice.

Table 4.2: Design alternatives and decisions

Decision	Alternative	Reason for the choice
AES-256-GCM	AES-256-CBC (proposal)	CBC gives no tamper detection by itself and needs a second mechanism. GCM gives confidentiality and authenticity in one operation [25], [23]
Random nonce for every file	Counter or fixed nonce	A new random 128-bit nonce makes reuse practically impossible and needs no shared state between runs
Per-file encryption	One archive per batch	Each file can be verified, decrypted and replaced on its own, and one damaged file does not block the batch
SHA-256 manifest of the plaintext	No manifest (rely on GCM tag only)	The manifest identifies the original image and lets another component confirm that a decrypted file is the one that was captured
Delete plaintext after encryption	Keep both copies	A readable copy would defeat the encryption
Deny-by-default RBAC	Allow-by-default with blocked actions	Unknown roles and unknown actions are refused, which follows least privilege [27]
Security classes independent of CARLA	Security code inside the capture script	Allows the same code to be tested in a separate audit notebook
Streaming hash in 4,096-byte blocks	Read the whole file into memory	Constant memory use for any file size

The first decision shaped the rest of the design. The proposal specified CBC, and the Privacy Integration thesis describes the dataset as encrypted with AES-256 in CBC mode [8]. The audited version of the security layer uses GCM. [CONFIRM: state which mode protected the 10,000-frame dataset delivered to the other components, and when the change to GCM was made.] The reason for the change is explained in Section 2.2: only an authenticated mode makes a modified file fail on decryption.

4.4 Detailed design

The component is written in Python. Figure 4.2 shows how a captured frame moves through it.

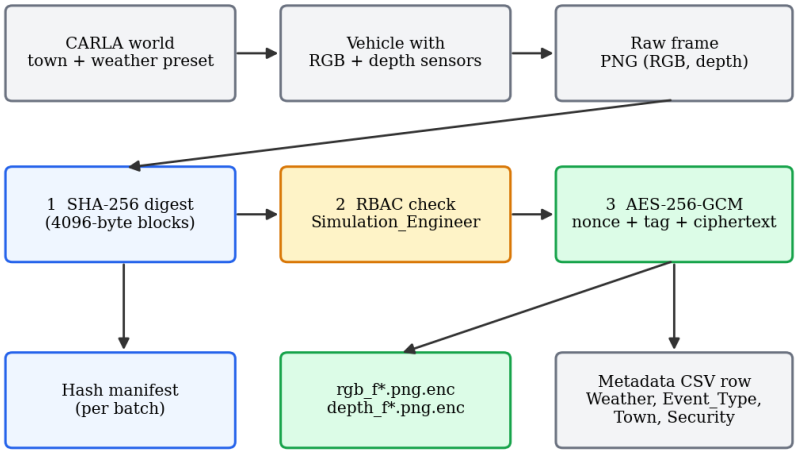


Figure 4.2: Internal data flow of the Dataset Generation component

Capture module. CARLA is started in a chosen town and a weather preset is applied. A vehicle with an RGB camera and a depth camera drives through the town. Frames are saved at a fixed interval. The module also assigns a driving-behavior label to each set, and it writes one metadata row per set.

Integrity module. The method `generate_sha256` reads each file in 4,096-byte blocks and returns the hexadecimal SHA-256 digest. The digest is written to the manifest of the batch. A mismatch found later marks the file as possibly changed.

Encryption module. The class `SecurityEngine` holds a 32-byte key. For each file it creates a new GCM cipher, which generates a fresh random nonce, encrypts and authenticates the data, and writes the nonce, the tag and the ciphertext to a file with the extension `.enc`. The plaintext file is then deleted. Figure 4.3 shows the layout of an encrypted file.

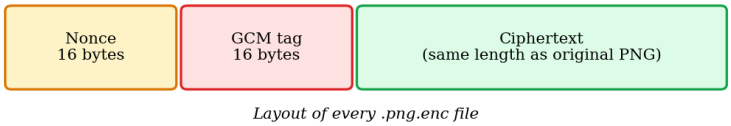


Figure 4.3: Layout of an encrypted dataset file

RBAC module. The class `RBACGateway` holds a dictionary from role to allowed operations. A request is granted only if the role exists and the operation is in its list. Table 4.3 shows the policy.

Table 4.3: RBAC role and permission matrix

Role	Permitted operations	Purpose
Simulation_Engineer	create, verify, encrypt	Produces and protects raw simulation output
Privacy_Engineer	verify	Checks integrity before privacy processing
ML_Researcher	read_processed	Uses processed datasets only

Threat model. The design assumes that the simulation computer is trusted and the storage is not. An attacker can read files from storage, change files in storage, or try to use a role without the needed permission. Table 4.4 maps each threat to the control and to the evidence in Chapter 6.

Table 4.4: Threat model and corresponding controls

Threat	Example	Control	Evidence
Tampering	Images in a batch are replaced	SHA-256 manifest; GCM tag	Test 5; re-hash by Risk and Compliance
Disclosure	Dataset files copied from storage	AES-256-GCM encryption; plaintext deleted	Tests 1 to 4; folder scan
Privilege misuse	An ML user reads raw output	RBAC deny by default	Role and permission matrix
Output linkage	Equal images show equal ciphertext	Fresh random nonce per file	Test 2
Silent corruption	Damaged file used for training	decrypt_and_verify fails with no output	Test 5

Security parameters

Table 4.5 lists the parameters of the security layer as implemented.

Table 4.5: Security parameters of the implementation

Parameter	Value
Hash function	SHA-256, 256-bit digest (64 hexadecimal characters), file read in 4,096-byte blocks
Cipher and key	AES with a 256-bit (32-byte) key
Mode	Galois/Counter Mode (GCM)
Nonce	16 bytes, random for every file, generated by PyCryptodome
Authentication tag	16 bytes (128 bits), checked before any plaintext is released
Stored layout	nonce, then tag, then ciphertext, in a file with the added extension .enc
Overhead per file	32 bytes
Key source	32-byte key supplied by the caller, or a random key from os.urandom if none is given
Plaintext handling	The plaintext file is deleted after encryption

Operation sequence for one file

Every captured file passes through the same sequence of operations. The sequence is deliberately short, so that it is easy to audit.

1. CARLA writes the raw PNG image to disk.
2. The pipeline asks the RBAC gateway whether the active role may create and encrypt data.
3. The SHA-256 digest of the file is computed and added to the manifest.
4. A new GCM cipher is created with a fresh random nonce, and the file is encrypted and authenticated.
5. The nonce, tag and ciphertext are written to the `.enc` file.
6. The plaintext file is deleted.
7. A record is added to the town metadata file.

Metadata schema

Each town has a metadata file in CSV format. One row describes one captured set. Table 4.6 lists the fields.

Table 4.6: Fields of the town metadata files

Field	Type	Meaning and example
RGB_File	Text	Name of the encrypted RGB image, for example rgb_f134256.png.enc
Depth_File	Text	Name of the encrypted depth image, for example depth_f134256.png.enc
Weather	Label	Weather setting of the capture, for example Clear_Day
Event_Type	Label	Driving behavior of the ego vehicle, for example Standard_Flow
Town	Label	CARLA map used, from Town01 to Town05
Security	Label	Protection applied to the files, SHA256+AES256 for every record

The metadata files contain only file names and labels. They do not contain image content, so they can be shared with the roles that need to select data without exposing the images.

4.5 Standards and professional practice

The design follows the standards and practices listed below.

- **Data protection by design.** Protection is applied when the data is created, not afterward. GDPR Article 32 asks for appropriate technical measures such as encryption and the ability to ensure ongoing integrity and confidentiality [3].
- **Cryptographic standards.** Hashing uses SHA-256 as defined in FIPS 180-4 [20]. Encryption uses AES as defined in FIPS 197 [21] in GCM mode as specified in NIST SP 800-38D [25].
- **Least privilege.** Role permissions follow the principle that each role receives only what its duties need [27], [26].
- **Key management guidance.** NIST SP 800-57 states that keys should be protected separately from the data they protect [34]. Section 7.5 discusses how far the current implementation meets this.
- **Engineering practice.** The security code is separated from the simulator code, tested in its own notebook and kept free of CARLA dependencies.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The component was developed and analyzed on a Windows laptop. Table 5.1 lists the main tools and their purpose. All of them are free and open-source.

Table 5.1: Development environment and technology stack

Tool	Purpose
CARLA Simulator [version], Python API	Driving scenes, weather, RGB and depth sensors
Python [version]	Main programming language
PyCryptodome [35]	AES-256-GCM encryption and decryption
Python hashlib	SHA-256 digests
pandas	Loading and combining the town metadata files
matplotlib and seaborn	Coverage charts for the dataset analysis and the audit charts
NumPy	Byte-frequency counts for the chi-square test
Jupyter Notebook	Security audit notebook and dataset analysis notebook
Hardware	[GPU model, CPU, RAM and storage of the capture machine]

5.2 Component implementation

The security layer is implemented as two Python classes. They do not depend on CARLA, so the same code runs in the capture pipeline and in the audit notebook.

Hashing. The method `generate_sha256` reads a file in 4,096-byte blocks and updates a SHA-256 object. The final digest is returned as a hexadecimal string and stored in the manifest.

```
def generate_sha256(self, file_path):
    sha256_hash = hashlib.sha256()
    with open(file_path, "rb") as f:
        for byte_block in iter(lambda: f.read(4096), b""):
            sha256_hash.update(byte_block)
    return sha256_hash.hexdigest()
```

Encryption. The method `encrypt_data` creates a GCM cipher with the 32-byte key, encrypts and authenticates the file contents in one call, writes the nonce, tag and ciphertext to a `.enc` file and deletes the plaintext.

```
def encrypt_data(self, file_path):
    cipher = AES.new(self.key, AES.MODE_GCM)
    with open(file_path, 'rb') as f:
        data = f.read()
    ciphertext, tag = cipher.encrypt_and_digest(data)
    enc_file_path = file_path + ".enc"
    with open(enc_file_path, "wb") as f:
        f.write(cipher.nonce + tag + ciphertext)
    os.remove(file_path)
    return enc_file_path
```

Decryption. The method `decrypt_data` reads the nonce and tag from the start of the file and calls `decrypt_and_verify`, which checks the tag before returning any plaintext. On failure

it prints the error, returns `False` and writes nothing. The encrypted file is removed only after a successful decryption.

```
def decrypt_data(self, enc_file_path):
    try:
        with open(enc_file_path, 'rb') as f:
            nonce = f.read(16)
            tag = f.read(16)
            encrypted_data = f.read()
        cipher = AES.new(self.key, AES.MODE_GCM, nonce=nonce)
        original_data = cipher.decrypt_and_verify(encrypted_data, tag)
        original_path = enc_file_path.replace(".enc", "")
        with open(original_path, "wb") as f:
            f.write(original_data)
        os.remove(enc_file_path)
        return True
    except Exception as e:
        print(f"Error during decryption: {e}")
        return False
```

Access control. The method `verify_access(user_role, permission)` of `RBACGateway` returns `True` only if the role is in the policy and holds the permission. Otherwise it prints a denial message and returns `False`.

```
self.roles = {
    "Simulation_Engineer": ["create", "verify", "encrypt"],
    "Privacy_Engineer": ["verify"],
    "ML_Researcher": ["read_processed"]
}
```

Capture and metadata. The capture pipeline starts CARLA in each town, applies a weather preset, drives the ego vehicle and records RGB and depth images. Each file is protected as described above. One row is written to the town metadata file for every captured set, with the columns RGB_File, Depth_File, Weather, Event_Type, Town and Security. The file names follow the pattern `rgb_f<frame>.png.enc` and `depth_f<frame>.png.enc`.

Storage overhead. Every encrypted file is exactly 32 bytes larger than its original, because of the 16-byte nonce and the 16-byte tag. If both images of each of the 10,000 captured sets are encrypted, this adds up to 20,000 files and 640,000 bytes (about 625 KB), which is small compared with the image data.

Implementation challenges

- **Encryption mode.** The proposal specified CBC. CBC cannot detect a modified encrypted file by itself [22], so the mode was changed to GCM, which detects tampering during decryption.
- **Key handling.** A group code review found keys written in source files before the repository was published. The keys were replaced and a regression check was added [8]. The audit notebook still uses a fixed test key so that results can be repeated, which is acceptable for testing but not for production (Section 7.5).
- **Testing without the simulator.** The security classes were written without any CARLA imports, so the audit could be run on any machine in seconds.

- **Dataset balance.** The behavior labels came out unbalanced. Vehicle_Turning has only 132 sets and High_Speed_Cruise exists only in Town04 (Section 6.2). The imbalance was found by the coverage analysis, and it affects what the other components can learn from the data.
- **Pedestrian visibility.** Privacy Integration reported that in 62.3% of frames labeled as containing a pedestrian or cyclist, the largest such object was smaller than about 8 x 8 pixels after resizing [8]. This is a property of the captured scenes, and it limited the tasks that could be trained on the data.

5.3 Integration with the other components

The component is connected to the three other components through files. Table 5.2 lists them.

Table 5.2: Files exchanged by the Dataset Generation component

File	Direction	Purpose
calibration_stats.json	From Privacy Integration	Privatized statistics used to configure the simulator
Encrypted frames (.png.enc)	To all three components	RGB and depth images protected with AES-256
Hash manifests	To Privacy Integration, Utility Evaluation, Risk and Compliance	SHA-256 digests to check files before use
Town metadata files (CSV)	To all three components	Weather, behavior label, town and security field per set
Encryption key	To Privacy Integration (separate channel)	Lets the other component decrypt the frames

Privacy Integration decrypted each frame, checked it against the SHA-256 manifest and then applied its label rule and data split [8]. Before any training started, Risk and Compliance re-hashed the decrypted dataset, and Utility Evaluation supplied clean labels through its poisoning gate [8]. The encryption key and the signing key of Privacy Integration traveled separately, so that no single key opened every control [8].

The privatized calibration values from Privacy Integration were used to configure the Town01 simulator profile. The Town01 profile matched the privatized file field by field [8]. The values included a vehicle density of 2.5139 per kilometer and a pedestrian density of 0.9153 per kilometer. The source values were reference statistics that this component had measured from earlier CARLA runs, so the data stayed inside the project.

5.4 Verification and software testing

The component was tested at three levels. The security audit tested the security classes on their own. The dataset analysis checked the generated metadata. The integration checks were carried out by the other components. Table 5.3 shows how each functional requirement was verified.

Table 5.3: Verification of the functional requirements

ID	Requirement	Verification	Status
FR-01	Several towns and weather settings	5 towns, 20 town and weather settings, 500 sets each	Passed
FR-02	RGB and depth capture with labels	10,000 metadata records with RGB, depth and behavior label	Passed
FR-03	SHA-256 digest and manifest	Manifest used by Risk and Compliance before training; no mismatch reported [8]	Passed
FR-04	AES-256 encryption of every file	Audit Tests 1 to 4; scan of 6,093 encrypted files with 0 readable	Passed
FR-05	Role-based access control	Policy implemented; denial tests not yet in the audit notebook	Partly verified
FR-06	Structured dataset with metadata	Five town metadata files with 2,000 records each	Passed
FR-07	Use of privatized calibration statistics	Town01 profile matched privatized file [8]	Passed
FR-08	Reject modified files	Audit Test 5: MAC check failed, no output written	Passed

FR-05 is marked as partly verified on purpose. The role policy exists and is applied in the pipeline, but the audit notebook does not yet include tests that show a denial for each wrong role and permission. Adding these tests is the first item of future work in Section 9.4.

Table 5.4 lists the test cases of the security audit, with the input, the steps, the expected result and the actual result.

Table 5.4: Test cases of the security audit

Test	Input	Steps	Expected	Actual
1	4 KB random bytes	Encrypt, decrypt, compare with the original	Identical bytes	Identical
2	One plaintext	Encrypt 30 times, collect nonces and ciphertexts, count unique values	30 and 30 unique	30 and 30
3	200,000 random bytes	Encrypt, count each byte value, compute chi-square	Below 293.25	272.59
4	Two 4 KB inputs, one bit apart	Encrypt both, XOR the ciphertexts, count differing bits	About 50%	50.0%
5	One encrypted file	XOR one byte with 0xFF, attempt decryption	Rejected, no output	MAC check failed
Scan	6,093 encrypted files	Read the first 8 bytes, compare with the PNG signature	No matches	0 matches

5.5 Deployment and operation

In the intended deployment, a simulation engineer runs the capture pipeline in each town with the Simulation_Engineer role. Each file is hashed, encrypted and written together with its metadata row. The encrypted folder and the manifests are then copied to the storage used by the other components. The key is delivered separately from the data, so that a copy of the

storage alone is not enough to read the images. A privacy engineer verifies batches before processing, and an ML researcher uses only processed datasets.

The audit notebook can be run at any time on any machine to repeat the five tests. A new town, weather setting or sensor can be added without changing the security classes, because they work on files and not on image content. The storage overhead is 32 bytes per file, so the layer scales with the number of files and not with their size.

6 RESULTS AND VALIDATION

6.1 Research results

This section reports the results of the five security audit tests and the scan of the real capture folder. Figure 6.1 shows the results of Tests 2, 3 and 4 and the folder scan.

Test 1: Round-trip correctness

A 4 KB file was encrypted and decrypted, and the recovered bytes were compared with the original.

Table 6.1: Round-trip correctness results

Metric	Result
Decryption reported success	True
Recovered bytes identical to original	True
Encryption time	14.914 ms
Decryption time	25.931 ms

The original data was restored exactly, so encryption does not corrupt dataset images. The times include reading and writing the file, and they come from a single run. They are small enough that the security layer does not noticeably slow the pipeline.

Test 2: Nonce uniqueness

The same plaintext was encrypted 30 times with the same key.

Table 6.2: Nonce uniqueness results

Metric	Result
Encryptions of identical plaintext	30
Unique nonces	30
Unique ciphertexts	30

All 30 nonces and all 30 ciphertexts were different. A person who sees two encrypted files cannot tell whether they hold the same image. The nonce is 128 bits long and random, so for 6,093 files the chance of any two nonces colliding is about 6,093 squared divided by 2 to the power 129, which is about 5 x 10^-32. Thirty runs is a small sample, so the test supports nonce uniqueness but does not prove it by itself. The guarantee comes from the random nonce generation in the library.

Test 3: Ciphertext byte distribution

A 200,000-byte file was encrypted and the frequency of each byte value in the ciphertext was counted.

Table 6.3: Ciphertext byte distribution results

Metric	Result
Ciphertext sample size	200,000 bytes
Expected count per byte value	781.25
Standard deviation of counts	28.84
Chi-square statistic (df = 255)	272.59
5% critical value (df = 255)	about 293.25
Approximate p-value	about 0.21

The statistic of 272.59 is close to the expected value of 255 and below the 5% critical value of about 293.25. The p-value of about 0.21 means this test cannot tell the ciphertext apart from uniformly random bytes. The notebook uses a pass threshold of 320, which is looser than the critical value, but the result also meets the stricter limit.

Test 4: Bit difference between encryptions

Two 4 KB inputs that differ in exactly one bit were encrypted, and the two ciphertexts were compared bit by bit.

Table 6.4: Bit difference results

Metric	Result
Bits compared	32,768
Bits different	16,384
Percentage different	50.0%

The ciphertexts differ in 50.0% of their bits, which is what unrelated random data gives. This result needs careful reading. GCM is built on counter mode, so a one-bit change in the plaintext encrypted under the same nonce would change only one ciphertext bit. The 50% appears here because each encryption uses its own fresh random nonce. The test therefore shows that similar files give unrelated-looking ciphertexts in practice. It should not be described as the avalanche effect of a block cipher.

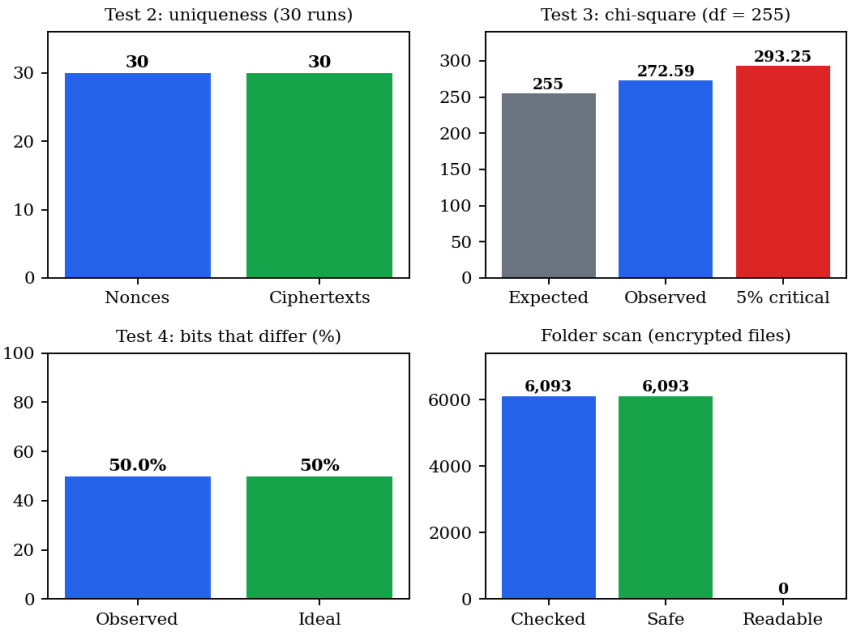


Figure 6.1: Results of audit Tests 2, 3 and 4 and the real folder scan

Test 5: Tamper detection

One byte inside an encrypted file was changed with an XOR of 0xFF, and decryption was attempted.

Table 6.5: Tamper detection results

Metric	Result
Decryption reported success	False
Error raised	MAC check failed
Output file produced	None

The modified file was rejected and no plaintext was written. This is the main practical gain from the change from CBC to GCM. A changed dataset file can no longer silently corrupt training data.

Scan of the real capture folder

The notebook scanned every file ending in `.png.enc` in the real Town01 capture folder and compared the first eight bytes with the PNG signature. A file that still starts with the PNG signature would be readable without the key.

Table 6.6: Real capture folder validation results

Metric	Result
Encrypted files checked	6,093
Files with no PNG header (safe)	6,093
Files with readable PNG header (vulnerable)	0

All 6,093 files were stored in encrypted form and none exposed a readable header. The scan shows that real output was protected. It does not measure cryptographic strength, which is the purpose of Tests 1 to 5.

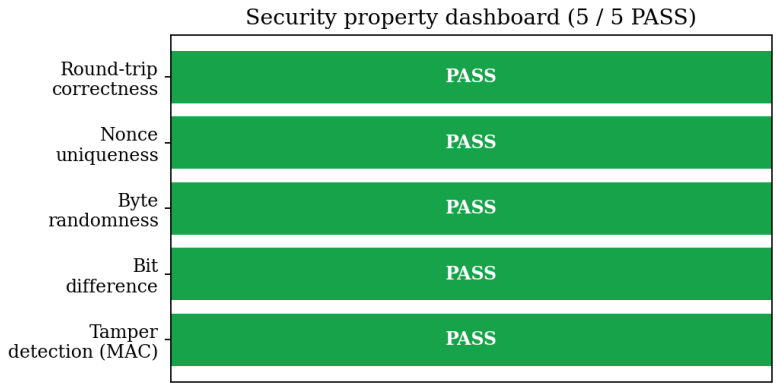


Figure 6.2: Security property dashboard (5 of 5 tests passed)

Table 6.7: Summary of audit results

Property	Key evidence	Outcome
Round-trip correctness	Bytes identical after decryption	Pass
Nonce uniqueness	30 of 30 unique nonces and ciphertexts	Pass
Byte randomness	Chi-square 272.59 (critical 293.25)	Pass
Bit difference	50.0% of bits differ	Pass
Tamper detection	MAC check failed on modified file	Pass
Real folder	0 of 6,093 encrypted files readable	Pass

6.2 Product and technical results

The component produced a protected dataset of 10,000 frames. The metadata files of the five towns were loaded into one table, and the notebook confirmed 2,000 records for each town and 10,000 records in total. Every record carries the security field `SHA256+AES256`.

Weather coverage

Each town was captured under four weather settings with 500 sets each, so the weather coverage is even inside each town. Figure 6.3 shows the distribution. Town01 covers day, overcast, rain and a stormy night. Town02 covers morning, rain, fog and sunset glare. Town03 and Town05 add urban storm, rain at night, dawn and mist, and Town04 adds highway sunset, mountain mist and wind-driven rain. Heavy_Rain appears in two towns and Clear_Noon in two towns, which gives 18 different labels for the 20 settings.

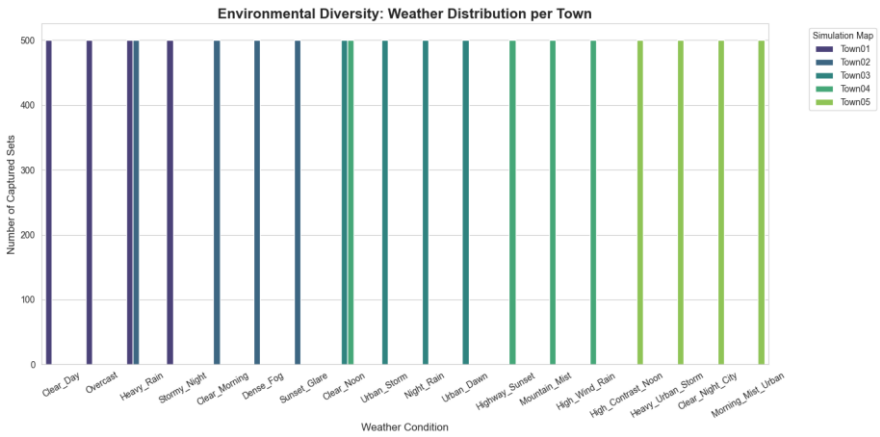


Figure 6.3: Weather distribution per town (number of captured sets)

Behavior coverage

The behavior labels are not balanced. Table 6.8 and Figure 6.4 show the counts over the whole dataset.

Table 6.8: Distribution of behavior labels over the dataset

Behavior label	Sets	Share of dataset
Standard_Flow	5,193	51.9%
Heavy_Braking	2,263	22.6%
Intersection_Nav	1,710	17.1%
High_Speed_Cruise	702	7.0%
Vehicle_Turning	132	1.3%
Total	10,000	100.0%

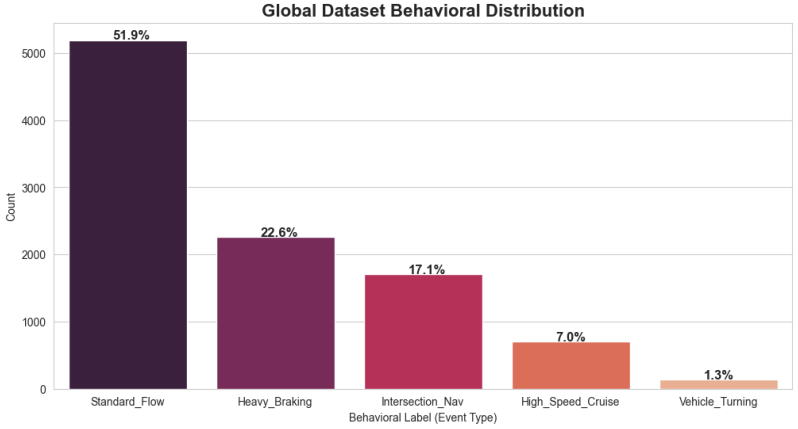


Figure 6.4: Global dataset behavior distribution

Standard_Flow makes up 51.9% of the dataset, and Heavy_Braking and Intersection_Nav add 22.6% and 17.1%. High_Speed_Cruise has 7.0% and Vehicle_Turning only 1.3%, or 132 sets. The largest class is about 39 times the size of the smallest.

Behavior by town

The heatmap in Figure 6.5 and Table 6.9 show how the behaviors are spread over the towns.

Table 6.9: Behavior labels by town (sets and share of the 2,000 sets of each town)

Town	Heavy braking	High-speed cruise	Intersection	Standard flow	Vehicle turning
Town01	790 (39.5%)	0 (0.0%)	101 (5.0%)	1089 (54.5%)	20 (1.0%)
Town02	922 (46.1%)	0 (0.0%)	173 (8.7%)	866 (43.3%)	39 (1.9%)
Town03	526 (26.3%)	0 (0.0%)	519 (25.9%)	892 (44.6%)	63 (3.1%)
Town04	25 (1.3%)	702 (35.1%)	380 (19.0%)	883 (44.1%)	10 (0.5%)
Town05	0 (0.0%)	0 (0.0%)	537 (26.9%)	1463 (73.2%)	0 (0.0%)

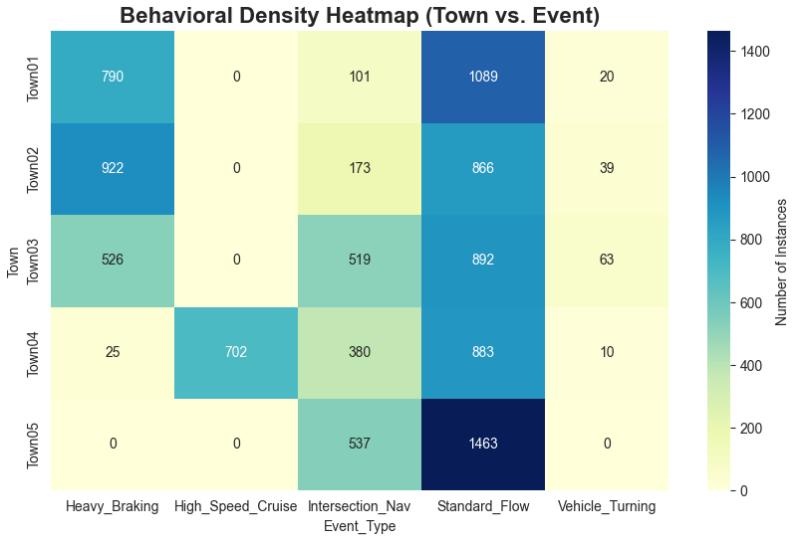


Figure 6.5: Behavioral density heatmap (town versus event)

Three points stand out. First, High_Speed_Cruise exists only in Town04, where it makes up 35.1% of the sets, which fits the highway layout of that map. Second, Town05 contains only two behaviors, Standard_Flow and Intersection_Nav, and no braking or turning. Third, Heavy_Braking is common in Town01 to Town03 (39.5%, 46.1% and 26.3%) and almost absent in Town04 (1.3%). The towns therefore differ not only in layout and weather but also in the behavior they contribute.

Diversity of behaviors

To compare the diversity of the towns, the normalized Shannon entropy of the behavior labels was calculated from the counts in Table 6.9. It is the entropy of the label distribution divided by the largest possible entropy for five labels. A value of 1 means that all five behaviors are equally common, and a value of 0 means that only one behavior occurs. Table 6.10 gives the results.

Table 6.10: Diversity of behavior labels by town

Town	Behaviors present	Largest class share	Normalized entropy
Town01	4	54.4%	0.556
Town02	4	46.1%	0.626
Town03	4	44.6%	0.727
Town04	5	44.1%	0.699
Town05	2	73.2%	0.361
All towns	5	51.9%	0.759

The whole dataset has a normalized entropy of 0.759, which is moderate. Town03 is the most balanced town at 0.727, and Town04 is close behind at 0.699 because it is the only town with all five behaviors. Town05 is the least diverse at 0.361, with 73.2% Standard_Flow and only two behaviors. The weather settings are perfectly even inside each town, so the unevenness lies entirely in the behaviors. A model trained on the full dataset will see many examples of normal driving and few examples of turning, and few examples of high-speed cruising outside one map.

Cost of the security layer

Table 6.11 summarizes the cost of the security layer.

Table 6.11: Cost of the security layer

Cost	Measured or derived value
Storage overhead per file	32 bytes (16-byte nonce and 16-byte tag)
Storage overhead for 20,000 files	640,000 bytes, about 625 KB (derived, if both images of every set are encrypted)
Encryption time, 4 KB file	14.914 ms, single run, including file input and output
Decryption time, 4 KB file	25.931 ms, single run, including file input and output
Keys	One 32-byte key for all files

If the time of the 4 KB test applied to every file, encrypting 20,000 files would take about five minutes. Real PNG images are larger than 4 KB, so the real time will be longer, and it should be measured on full-size batches. Even so, the cost is small compared with the time needed to run the simulation itself.

6.3 User or stakeholder validation

The component had no external end users during the project. Validation came from the groups that used or reviewed its outputs.

Supervisors. The supervisor and co-supervisor reviewed the work at the project milestones and guided the choice of controls and the scope of the component.

Privacy Integration as a user of the data. Privacy Integration decrypted the dataset, checked each file against the manifest and trained a classifier on it [8]. Its use confirmed that the

encrypted files and manifests can be consumed by another component. Its work also gave useful feedback on the data. In 62.3% of frames labeled as containing a pedestrian or cyclist, the largest such object was smaller than about 8 x 8 pixels after resizing, and 21.5% of the frames had to be excluded because they contained only small targets [8]. These figures show that pedestrian coverage in the captured scenes is thin.

Utility Evaluation. Utility Evaluation trained a detector on the synthetic data and on real KITTI images. The synthetic model reached 10.46% of the real-data mAP50, so the group target of staying within 15% of the real-data baseline was not met [8]. This result concerns the realism and variety of the whole dataset and not the security layer, and it is discussed in Section 7.5.

6.4 Integration results

Table 6.12 summarizes how the other components used the output of this component, as reported in the Privacy Integration thesis [8].

Table 6.12: Integration results reported by the other components

Interface	What was checked	Outcome
Encrypted frames and key	Privacy Integration decrypted frames for training	Worked
Hash manifest	Frames checked against the manifest before use	No mismatch reported
Hash manifest	Risk and Compliance re-hashed the decrypted dataset before training	Part of the pipeline run
calibration_stats.json	Town01 simulator profile compared with the privatized file	Matched field by field
Metadata and labels	Privacy Integration split 10,000 frames into 796 positive, 7,052 negative and 2,152 excluded	Used for training
Dataset for detector	Utility Evaluation trained a detector on the synthetic frames	10.46% of real-data mAP50

The integration shows that the security layer did not block the other components. Files could be decrypted by the party that held the key, and they could be verified against the manifest. It also shows a weakness that the security layer cannot fix: the usefulness of the dataset depends on how well the scenes are designed, as discussed in Chapter 7.

6.5 Summary of findings

- The audit tests passed: encryption restored the data exactly, 30 of 30 nonces and ciphertexts were different, the chi-square statistic was 272.59 against a 5% critical value of about 293.25, 50.0% of ciphertext bits differed, and a changed file was rejected with a MAC check failure.
- All 6,093 encrypted files in the real capture folder had no readable PNG header.

- The encryption cost 32 bytes per file, and a 4 KB file took 14.9 ms to encrypt and 25.9 ms to decrypt.
- The dataset has 10,000 frames in five towns and 20 weather settings with 500 sets each, so weather coverage is even.
- Behavior coverage is unbalanced: Standard_Flow is 51.9% and Vehicle_Turning only 1.3%, High_Speed_Cruise exists only in Town04, and Town05 has only two behaviors.
- The other components could decrypt, verify and use the dataset, and their work showed that pedestrian coverage and overall utility need improvement.

7 DISCUSSION

7.1 Interpretation of findings

The results answer the main research question with a yes, within clear limits. For RQ1, every captured frame was hashed, encrypted and covered by the access policy at capture time, and the simulator was used in the normal way, because the security classes work on files after they are written. For RQ2, the encryption layer passed all five audit tests. The data came back exactly, identical inputs never gave identical outputs, the ciphertext was statistically consistent with random bytes, and a changed file was rejected. For RQ3, the dataset covers five towns and 20 weather settings evenly, but it is unbalanced in behavior, and the security layer costs 32 bytes and a few milliseconds per file.

The size of the effect needs a careful reading. The audit shows that no weakness was found in the properties tested. It does not show that the system cannot be broken. A passing statistical test only shows that no pattern was seen. The strength of the design comes mainly from the standard primitives it uses, AES-256, GCM and SHA-256, and from using them in the way their standards describe. The tests mostly show that the implementation did not break that design, for example by reusing a nonce or by skipping the tag check.

The choice of mode mattered most. With CBC, a modified file may decrypt into damaged data, and the pipeline would need a second check to notice. With GCM, the tag check is part of decryption. This also explains why the SHA-256 manifest is still kept. The tag protects the encrypted file, while the manifest identifies the original plaintext and can be checked by another component after decryption.

7.2 Comparison with literature baselines and existing solutions

The design agrees with the guidance in the literature. Studies of real attacks on GCM show that nonce reuse is the main danger [14], [15]. Generating a fresh random nonce for every file and testing that no two are equal addresses that danger directly. Work on CBC shows that unauthenticated encryption can be attacked through its padding behavior [22], and that combining encryption and authentication correctly is not trivial [23]. Using an authenticated mode avoids both problems.

The security surveys of autonomous driving recommend integrity protection, encryption and access control, but they stay at the level of recommendations [6], [7], [9]. This thesis adds a working control inside a synthetic data pipeline with measured evidence. Compared with plain disk or cloud encryption, the component protects each file at the moment of capture, records a digest for each file and applies a role policy to pipeline operations. Disk encryption does none of these things on its own. It is cheaper to set up, however, and the two can be used together.

The component also fills a gap that privacy-focused work leaves open. Differential privacy limits what a model or dataset reveals [29], [30], and the work of Stadler et al. shows why such

formal protection is needed for synthetic data [28]. It does not stop someone changing the files. The security layer and the privacy layer address different threats, and the project needs both.

7.3 Research contribution

The main research contribution is a measured evaluation of a security layer inside a synthetic AV data pipeline. The evaluation combines five tests, each aimed at a specific weakness, with a scan of real output. A second contribution is the careful reading of the bit-difference test. In GCM, the 50% result comes from fresh nonces and not from diffusion inside the cipher, and reporting this avoids a common misreading. A third contribution is the coverage analysis of the generated dataset. It shows that even weather coverage can hide an uneven spread of behaviors, and that a security layer does not make a dataset more useful.

7.4 Product contribution

The product contribution is the Dataset Generation component. It captures labeled CARLA frames in five towns, protects every file, enforces role-based access, and delivers the dataset and its metadata to the other three components. The security classes are independent of CARLA and can be reused by other pipelines. The audit notebook lets any team repeat the checks in seconds, and the town metadata files make the dataset easy to analyze.

7.5 Limitations and threats to validity

Internal validity. The audit tests used pseudo-random bytes and not real PNG images, and the nonce test used 30 runs. The bit-difference test measures the effect of fresh nonces and does not measure diffusion inside the cipher. The timings come from single runs on one machine. The audit notebook covers the encryption layer. SHA-256 manifest verification, including a test with a modified file, and denial cases of the RBAC gateway are part of the pipeline but are not yet in the notebook.

Construct validity. Statistical tests show that no weakness was seen. They are not a formal security proof and not a replacement for cryptanalysis. Only a chi-square test was used for randomness, while a full suite such as NIST SP 800-22 [31] would examine more patterns. The folder scan detects only unencrypted PNG headers.

Key management. The audit notebook uses a fixed test key so that results can be repeated. A key that is written in code is not acceptable in production, and the group code review found a similar problem in the repository before it was published [8]. NIST guidance says that keys should be protected separately from the data they protect and managed through their life cycle [34]. In the project, the key was shared with Privacy Integration on a separate channel, but the component has no key management service, no key rotation and a single key for all files. A leak of that key would expose every file. [CONFIRM: describe how the production key is loaded, for example from an environment file kept out of version control.]

Access control. The RBAC gateway receives roles as plain strings, and it has no user authentication and no persistent audit log. The policy has no permission for decryption, so the decision on who may decrypt is left to whoever holds the key. The roles are enforced by the pipeline wrapper, and a person who has direct access to the files and the key can bypass the wrapper.

Documentation of the encryption mode. The proposal and the Privacy Integration thesis describe the dataset as encrypted with CBC [8], while the audited security layer uses GCM. [CONFIRM: if the delivered dataset was encrypted before the change to GCM, state this here and note that files in CBC format are protected for confidentiality but not for tamper detection by the encryption layer alone.]

External validity. All data comes from one simulator on one machine. The folder scan covered 6,093 files from the Town01 capture folder, and the full dataset holds 10,000 frames, so the scan did not cover every file. The results show that the layer works as designed. They do not show how it behaves at much larger scale.

Dataset limitations. The dataset is unbalanced in behavior. Vehicle_Turning has 132 sets, High_Speed_Cruise is found only in Town04 and Town05 has two behaviors. Pedestrians are often small in the images, which limited the tasks that Privacy Integration could train [8]. The metadata schema records RGB and depth files, and semantic segmentation, which the proposal also listed, is not part of the metadata [CONFIRM]. The calibration statistics were reference values from CARLA runs and not values from a real dataset such as Waymo. Utility Evaluation found that a detector trained on the synthetic data reached 10.46% of the real-data mAP50, so the group target of 15% was not met [8]. The cause is not the security layer, because encryption is lossless (Test 1). It lies in the realism and variety of the generated scenes.

Table 7.1 summarizes the main limitations, their effect and the suggested remedy.

Table 7.1: Limitations, their effect and suggested remedies

Limitation	Effect	Suggested remedy
One fixed key for all files	A leak of the key exposes every file	Key management service, rotation, separate keys per batch or role [34]
Roles are strings, no user authentication	A caller can claim any role	Authenticate users before role checks
No persistent access log	Access cannot be audited afterward	Tamper-evident access log
Audit has no SHA-256 or RBAC denial tests	FR-03 and FR-05 are only partly evidenced	Add the two test groups to the notebook
Random-byte inputs and 30 nonce samples	Statistical support only	Real PNG files, larger samples, NIST test suite [31]
Unbalanced behavior labels	Few turning and braking examples in some towns	Add targeted scenarios
Small pedestrians in images	Limits pedestrian tasks	Scenes with closer, larger pedestrians
CBC and GCM described in different documents	Possible mismatch between documents	Confirm the delivered mode and align the documents

7.6 Implications

For practitioners, the results suggest that a synthetic dataset should be protected at generation and not afterward. The cost is small: 32 bytes and a few milliseconds per file. An authenticated mode such as GCM should be preferred to unauthenticated modes, and nonce uniqueness should be tested and not assumed. A security layer should be tested with repeatable checks that another party can run.

For the design of synthetic data pipelines, security and data quality are separate goals. A dataset can be fully protected and still fail to be useful if its scenes lack variety, as the behavior imbalance and the detector result show. Teams should therefore review coverage together with security. For auditors and data controllers, a per-file manifest, authenticated encryption and a role policy give evidence that can be checked without trusting the team that produced the data.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The component gives a team that builds perception models with simulated data a way to protect and share its datasets with evidence. Every file is hashed and encrypted at capture, access follows a role policy, and the whole layer can be tested with a notebook. Unlike approaches where security is added after generation, the protection starts when the data is created.

The project proposal identified the following groups of users:

- Autonomous vehicle researchers and AI developers, who need training data they can trust and share.
- Simulation engineers, who generate the data and need a layer that does not disturb capture.
- Cybersecurity engineers, who configure, monitor and audit the controls and the keys.
- Technology companies and the automotive industry, which want to improve models without collecting real data.

8.2 Market and competitive position

The target market is teams that use simulation for perception training, such as autonomous vehicle start-ups, automotive suppliers, university labs and simulation service providers. Many of them exchange datasets with partners and must show that the data was handled securely.

Competing approaches each cover only part of the need. Simulators such as CARLA [4] produce data but do not protect it. Storage and cloud encryption protect files but are applied after generation and give no per-file integrity record or role policy for pipeline steps. Anonymization and privacy tools protect what the data reveals but not the files themselves. This component combines hashing, authenticated encryption and role-based access at the point of generation, and it can be added to an existing CARLA workflow as a security extension.

8.3 Business or sustainability model

The proposal described three revenue paths. The solution could be sold as a software platform on a subscription basis. It could be offered as a dataset generation service, with pricing by the number of scenarios, dataset size and compute, or as a pay-per-dataset service for one-time needs. It could also be integrated into existing simulation platforms as a security extension. An open-source core with paid integration, support and audit services is a realistic route, because it builds trust in the method and keeps it open to review.

Table 8.1 shows the budget estimate of the whole group from the project proposal. The largest items are cloud compute and paper access. They belong to the group as a whole and mostly to the privacy experiments. The security layer of this component needs no paid software.

Table 8.1: Project budget estimate from the proposal (whole group)

Item	Cost (LKR)	Note
Cloud GPU compute (about 200 hours)	35,000	Mainly for differential-privacy experiments
Cloud storage (500 GB)	5,000	Dataset batches, encrypted files and logs
Opacus and TensorFlow Privacy	0	Open-source
Python cryptographic libraries (hashlib, PyCryptodome)	0	Open-source
Research paper access	8,000	Paywalled papers
Human effort	0	Covered by the academic project
Total	48,000	Estimate from the proposal

The running costs are low. CARLA, PyCryptodome and the analysis libraries are open-source. The main costs are cloud compute for large simulations, storage for datasets and staff time. The project proposal estimated the budget of the whole group at about LKR 48,000, mostly for cloud GPU time, cloud storage and research papers. The security layer itself adds only 32 bytes per file, so it does not increase storage or compute cost in a significant way.

8.4 Scalability intellectual property and compliance

Scalability. The security layer works on files, and its cost grows with the number of files and not with image content. It can be applied to more towns, more weather settings and more sensors without changes. Large-scale generation would need parallel simulation, a proper key management service and automated logging of access decisions.

Intellectual property. The component builds on open-source tools, and any release must respect their licences and keep their notices. The licences of CARLA and its assets need to be checked before commercial use. The code written for this component belongs to the author and the project, subject to the rules of the university. The proposal suggests protecting the integration of security and generation with copyright or, where possible, other forms of protection. Datasets that reuse benchmark data such as KITTI would also need to follow the licence of that dataset.

Compliance. The component produces technical evidence for compliance. Encryption and integrity protection support GDPR Article 32 on security of processing [3], and the evidence can be mapped to the NIST Privacy Framework [36]. The decision about legal compliance stays with the data controller. A product would also have to state the limits of the layer clearly, such as the key management limits in Section 7.5.

8.5 Contribution to the Sustainable Development Goals

The component contributes most directly to two Sustainable Development Goals [37].

- **SDG 9: Industry, Innovation and Infrastructure.** Target 9.5 asks for stronger research and technological capability. The component gives developers a tested way to share synthetic driving data securely, which supports innovation in the automotive sector.
- **SDG 16: Peace, Justice and Strong Institutions.** The component supports accountable handling of data, because it gives auditors evidence they can verify, and it protects data that may later relate to real people.

The link to road safety under SDG 3 and SDG 11 is indirect. Safer sharing of training data may help perception models improve, but this thesis did not measure any effect on road safety. There are also possible negative effects. Encryption can give a false sense of security if users ignore the limits of key handling. A lost key would make the dataset permanently unreadable. Large simulations use energy. These risks can be reduced by publishing the limits together with the evidence, by backing up keys under a proper key management policy, and by capturing only the data that is needed.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This research built a security layer into the synthetic data generation pipeline of an autonomous vehicle training framework and tested it with repeatable checks. The Dataset Generation component captures CARLA frames in five towns under 20 weather settings, computes a SHA-256 digest for every file, encrypts every file with AES-256-GCM and applies a role-based access policy. It produced a protected dataset of 10,000 frames with a metadata file for each town.

The security audit found no failures in five tests. Encryption restored the data exactly, 30 of 30 encryptions of the same input gave different nonces and ciphertexts, the chi-square statistic of 272.59 was below the 5% critical value of about 293.25, 50.0% of ciphertext bits differed between two encryptions, and a modified file was rejected with a MAC check failure. All 6,093 encrypted files in the real capture folder had no readable header. The cost was 32 bytes and a few milliseconds per file. The main finding is that integrity, confidentiality and role-based access can be embedded at generation time with standard tools and a small overhead. The other components could decrypt, verify and use the dataset.

The work also shows limits. The key management of the pipeline is not production grade, the RBAC tests are incomplete, and the dataset is unbalanced in behavior and thin in pedestrian coverage. A secure dataset is not automatically a useful one.

9.2 Achievement of objectives

Table 9.1 compares the results with the objectives in Section 1.5.

Table 9.1: Achievement of objectives

Objective	Outcome	Status
Configure CARLA capture across towns and weather	5 towns, 20 weather settings, RGB and depth frames	Achieved
Generate a diverse, labeled dataset	10,000 labeled frames; even weather coverage; behavior labels unbalanced	Partly achieved
SHA-256 digest and manifest for every file	Digest and manifest used in the pipeline; no mismatch reported by other components	Achieved
AES-256 authenticated encryption of every file	AES-256-GCM; all audit tests passed; 0 of 6,093 files readable	Achieved
RBAC for three roles	Policy implemented with deny by default; denial tests not yet in audit	Partly achieved
Independent audit of the security layer	Five tests plus a folder scan, all passed	Achieved
Integrate with the other components	Privatized statistics used; dataset decrypted, verified and used by all three components	Achieved

9.3 Individual research and product contributions

The individual research contributions of this thesis are as follows:

- A measured evaluation of a security layer inside a synthetic AV data pipeline, with five tests aimed at specific weaknesses and a scan of real output.
- A careful reading of the bit-difference result for GCM, which separates the effect of fresh nonces from cipher diffusion.
- A coverage analysis of the generated dataset that shows even weather coverage but uneven behavior coverage.

The individual product contributions are as follows:

- The `SecurityEngine` and `RBACGateway` classes, independent of CARLA.
- The encrypted 10,000-frame dataset with manifests and town metadata files.
- The audit notebook and the dataset analysis notebook.
- The interfaces that connect the component to the other three components.

9.4 Future directions

The future work is listed in order of priority.

1. **Complete the audit.** Add tests for SHA-256 manifest verification, including a modified file, and for RBAC denials of every wrong role and permission.
2. **Key management.** Move keys to a key management service, rotate them, use separate keys per batch or per role and keep them apart from the data [34].
3. **Stronger access control.** Add user authentication, a persistent and tamper-evident access log and an explicit decryption permission.
4. **Better scenes.** Add scenarios with more pedestrians and cyclists at larger image scale, more turning and braking events in every town and balanced behaviors, so that the dataset becomes more useful.
5. **More thorough randomness testing.** Use real PNG images, larger nonce samples and the NIST statistical test suite [31].
6. **Scale and performance.** Benchmark hashing and encryption on full-size batches and in parallel simulations.
7. **Real calibration data.** Take the calibration statistics from a real dataset such as Waymo [2], and repeat the dataset generation at more than one calibration budget.

REFERENCES

[1] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2012, pp. 3354-3361.

[2] P. Sun et al., "Scalability in perception for autonomous driving: Waymo Open Dataset," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 2446-2454.

[3] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)," Off. J. Eur. Union, vol. L 119, pp. 1-88, May 2016.

[4] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in Proc. 1st Annu. Conf. Robot Learn. (CoRL), ser. Proc. Mach. Learn. Res., vol. 78, 2017, pp. 1-16.

[5] T. Sun et al., "SHIFT: A synthetic driving dataset for continuous multi-task domain adaptation," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022.

[6] C. Gao, G. Wang, W. Shi, Z. Wang, and Y. Chen, "Autonomous driving security: State of the art and challenges," IEEE Internet Things J., vol. 9, no. 10, pp. 7572-7595, May 2022.

[7] Y. Liao et al., "Data security in autonomous driving: Multifaceted challenges of technology, law, and social ethics," World Electr. Veh. J., vol. 16, no. 1, art. 6, 2025.

[8] S. A. S. D. Priyadarshi, "Privacy Integration: Individual thesis (R26-CS-010, IT22309556)," Sri Lanka Inst. Inf. Technol., Colombo, Sri Lanka, Oct. 2026.

[9] G. Lippi, M. Aljawarneh, Q. Al-Na'amneh, R. Hazaymih, and L. D. Dhomeja, "Security and privacy challenges and solutions in autonomous driving systems: A comprehensive review," J. Cyber Secur. Risk Audit., vol. 2025, no. 3, pp. 23-41, 2025.

[10] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," in Proc. 29th Int. Conf. Mach. Learn. (ICML), 2012, pp. 1467-1474.

[11] T. Gu, B. Dolan-Gavitt, and S. Garg, "BadNets: Identifying vulnerabilities in the machine learning model supply chain," 2017, arXiv:1708.06733.

[12] S. I. Nikolenko, "Synthetic data for deep learning," 2019, arXiv:1909.11512.

[13] C. Xie, Z. Cao, Y. Long, D. Yang, D. Zhao, and B. Li, "Privacy of autonomous vehicles: Risks, protection methods, and future directions," 2022, arXiv:2209.04022.

[14] N. Ferguson, "Authentication weaknesses in GCM," comments submitted to NIST, 2005.

[15] H. Böck, A. Zauner, S. Devlin, J. Somorovsky, and P. Jovanovic, "Nonce-disrespecting adversaries: Practical forgery attacks on GCM in TLS," in Proc. 10th USENIX Workshop Offensive Technol. (WOOT), 2016.

[16] M. Cordts et al., "The Cityscapes dataset for semantic urban scene understanding," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 3213-3223.

[17] G. Ros, L. Sellart, J. Materzynska, D. Vazquez, and A. M. Lopez, "The SYNTHIA dataset: A large collection of synthetic images for semantic segmentation of urban scenes," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 3234-3243.

[18] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," in Field and Service Robotics. Cham, Switzerland: Springer, 2018, pp. 621-635.

[19] J. Tremblay et al., "Training deep networks with synthetic data: Bridging the reality gap by domain randomization," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW), 2018, pp. 969-977.

[20] Secure Hash Standard (SHS), FIPS PUB 180-4, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2015.

[21] Advanced Encryption Standard (AES), FIPS PUB 197, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2001.

[22] S. Vaudenay, "Security flaws induced by CBC padding: Applications to SSL, IPSEC, WTLS...," in Advances in Cryptology - EUROCRYPT 2002, ser. Lecture Notes Comput. Sci., vol. 2332. Berlin, Germany: Springer, 2002, pp. 534-545.

[23] M. Bellare and C. Namprempre, "Authenticated encryption: Relations among notions and analysis of the generic composition paradigm," in Advances in Cryptology - ASIACRYPT 2000, ser. Lecture Notes Comput. Sci., vol. 1976. Berlin, Germany: Springer, 2000, pp. 531-545.

[24] D. A. McGrew and J. Viega, "The security and performance of the Galois/Counter Mode (GCM) of operation," in Progress in Cryptology - INDOCRYPT 2004, ser. Lecture Notes Comput. Sci., vol. 3348. Berlin, Germany: Springer, 2004, pp. 343-355.

[25] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2007.

[26] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," Computer, vol. 29, no. 2, pp. 38-47, Feb. 1996.

[27] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proc. IEEE, vol. 63, no. 9, pp. 1278-1308, Sep. 1975.

[28] T. Stadler, B. Oprisanu, and C. Troncoso, "Synthetic data: Anonymisation groundhog day," in Proc. 31st USENIX Secur. Symp., 2022, pp. 1451-1468.

[29] M. Abadi et al., "Deep learning with differential privacy," in Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), 2016, pp. 308-318.

[30] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," Found. Trends Theor. Comput. Sci., vol. 9, no. 3-4, pp. 211-407, 2014.

[31] A. Rukhin et al., "A statistical test suite for random and pseudorandom number generators for cryptographic applications," NIST Special Publication 800-22 Rev. 1a, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2010.

[32] S. Hernan, S. Lambert, T. Ostwald, and A. Shostack, "Uncover security design flaws using the STRIDE approach," MSDN Mag., Nov. 2006.

[33] CARLA Team, "CARLA documentation: Maps," [Online]. Available: https://carla.readthedocs.io

[34] E. Barker, "Recommendation for key management: Part 1 - General," NIST Special Publication 800-57 Part 1 Rev. 5, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2020.

[35] PyCryptodome Project, "PyCryptodome documentation," [Online]. Available: https://www.pycryptodome.org

[36] Nat. Inst. Standards Technol., "NIST Privacy Framework: A tool for improving privacy through enterprise risk management, Version 1.0," U.S. Dept. Commerce, Gaithersburg, MD, USA, 2020.

[37] United Nations, "Transforming our world: The 2030 Agenda for Sustainable Development," General Assembly Resolution A/RES/70/1, 2015.

GLOSSARY

Authenticated encryption (AEAD). Encryption that also proves the data was not changed. A change makes decryption fail.

Authentication tag. A 16-byte value that GCM computes over the ciphertext and checks during decryption.

Nonce. A number used once per encryption. In GCM it must never repeat under the same key.

Semantic security. The property that an observer learns nothing from a ciphertext, including whether two files are equal.

SHA-256 digest. A 256-bit fingerprint of a file. Any change to the file changes the digest.

Hash manifest. A list of file names and their SHA-256 digests that is kept next to a batch.

Cipher Block Chaining (CBC). A block cipher mode that gives secrecy but no built-in detection of changes.

Galois/Counter Mode (GCM). A block cipher mode that encrypts with a counter and adds an authentication tag.

Role-based access control (RBAC). A policy in which permissions are given to roles and users act through roles.

Least privilege. The rule that each role gets only the permissions it needs.

Deny by default. The rule that anything not explicitly allowed is refused.

Chi-square test. A statistical test that compares observed counts with expected counts. Here it checks whether byte values in a ciphertext are evenly spread.

Ego vehicle. The simulated vehicle that carries the sensors and drives through the town.

Captured set. One RGB image and one depth image of the same frame, with one metadata record.

Behavior label. The driving behavior of the ego vehicle when a set was captured, for example Heavy_Braking.

Plaintext. The original, unencrypted data.

APPENDICES

Appendix A Extended results

This appendix gives the raw audit values and the data behind the coverage figures in more detail than Chapter 6. Table A.1 lists the raw values of the five audit tests.

Table A.1: Raw values of the security audit

Measure	Value
Round-trip: recovered bytes identical	True
Round-trip: encryption time (4 KB)	14.914 ms
Round-trip: decryption time (4 KB)	25.931 ms
Nonce test: encryptions / unique nonces / unique ciphertexts	30 / 30 / 30
Byte distribution: sample size	200,000 bytes
Byte distribution: expected count per value	781.25
Byte distribution: standard deviation of counts	28.84
Byte distribution: chi-square (df = 255)	272.59
Byte distribution: 5% critical value / p-value	about 293.25 / about 0.21
Bit difference: bits compared / bits different	32,768 / 16,384 (50.0%)
Tamper test: result	MAC check failed; no output file
Folder scan: files checked / safe / readable	6,093 / 6,093 / 0

Table A.2 shows the first records of the Town01 metadata file. It illustrates the metadata schema. The file names of the first five records differ by 16 in the frame number.

Table A.2: First five records of the Town01 metadata file

RGB_File	Depth_File	Weather	Event_Type	Town	Security
rgb_f134256.png.enc	depth_f134256.png.enc	Clear_Day	Standard_Flow	Town01	SHA256+AES256
rgb_f134272.png.enc	depth_f134272.png.enc	Clear_Day	Standard_Flow	Town01	SHA256+AES256
rgb_f134288.png.enc	depth_f134288.png.enc	Clear_Day	Standard_Flow	Town01	SHA256+AES256
rgb_f134304.png.enc	depth_f134304.png.enc	Clear_Day	Standard_Flow	Town01	SHA256+AES256
rgb_f134320.png.enc	depth_f134320.png.enc	Clear_Day	Standard_Flow	Town01	SHA256+AES256

Table A.3 gives the number of sets of each behavior label in each town, which is the data behind Figure 6.5.

Table A.3: Number of sets per behavior label and town

Town	Heavy_Braking	High_Speed_Cruise	Intersection_Nav	Standard_Flow	Vehicle_Turning
Town01	790	0	101	1,089	20
Town02	922	0	173	866	39
Town03	526	0	519	892	63
Town04	25	702	380	883	10
Town05	0	0	537	1,463	0
Total	2,263	702	1,710	5,193	132

Table A.4 gives the expected outcome of every role and permission pair under the access policy. The audit notebook does not yet test each of these cases. They are the planned denial tests.

Table A.4: Expected outcome of each role and permission pair

Role	create	verify	encrypt	read_processed	Other
Simulation_Engineer	Allow	Allow	Allow	Deny	Deny
Privacy_Engineer	Deny	Allow	Deny	Deny	Deny
ML_Researcher	Deny	Deny	Deny	Allow	Deny
Unknown role	Deny	Deny	Deny	Deny	Deny

Table A.5 describes the steps of the audit notebook so that another person can repeat it.

Table A.5: Steps of the security audit notebook

Step	Action	Output
1	Define SecurityEngine and RBACGateway (same code as the pipeline)	Classes ready, test key set
2	Run Test 1 on a 4 KB random file	Match result and timings
3	Run Test 2: encrypt one plaintext 30 times	Counts of unique nonces and ciphertexts
4	Run Test 3 on a 200,000-byte file	Byte counts, chi-square statistic
5	Run Test 4 on two inputs one bit apart	Number and share of differing bits
6	Run Test 5: change one byte and decrypt	MAC check result, no output file
7	Scan the real encrypted folder	Counts of safe and readable files
8	Save charts and results	audit_results.json and image files

Appendix B Academic integrity and AI use disclosure

Generative AI was used while preparing this thesis, in line with the module policy on AI use.

Tool. Claude, a large language model by Anthropic.

Use. The tool was used to help structure the thesis according to the module template and a peer thesis, to draft and edit text based on the author's proposal, notebooks and project files, to create the diagrams and charts from the author's own results, to check references against published records, and to compute derived values such as the chi-square critical value, the p-value and percentages.

Verification. The author checked the numbers, citations and claims in the thesis against the notebook outputs and the cited sources.

Responsibility. The design of the component, the experiments, the results and the conclusions are the responsibility of the author.